

Bridge Gap: Complexity_Simplified_p to Complexity_Simplified_N

Puzzle 135 / secp256k1 research note. Generated 2026-06-09.
This document records what the p-side file completes, what the N-side file assumes, and the missing closure (Lambda_N, range-variable delta, carry, cubic aggregate).

1. Executive summary

Complexity_Simplified_p closes the x-bridge: $Px_i / rx_i = \text{Lambda} \pmod p$ via three cube-root branches (x^3). y follows the same make-up but with only two square-root branches (y^2), coupled by reflection ($\text{lambda}_y + \text{lambda}_{-y} = N$). N-file must use $\text{Lambda}_N = \text{Lambda} + \text{GAP}$ and range-variable defect(d), not a blind copy of p-side Lambda or Lambda^3 .

2. Side-by-side completion map

Layer	p-file	N-file
Root	$n^3 = N \pmod p$	$p^3 = p \pmod N$
Normalize x	n^3 roots -> 3-way Latin square	p^3 roots -> no collapse
Normalize y	y^2 -> 2 branches (+/- only)	reflection pair (not 3-way)
Collapse x	CP1, CR1 -> Lambda	MISSING CQ1, Cq1
Point bridge x	$Px/rx = \text{Lambda} \pmod p$ OK	$Qx = \text{Lambda}_N * qx$ OK (scaled)
Point bridge y	$Py/ry = \text{lambda}_y$ (2-fold)	$\text{lambda}_y + \text{lambda}_{-y} = N$
Power law	$\text{lam}_y^2 = (Px^3 + 7)/(rx^3 + 7)$ OK	Ratio + naive forms FAIL mod N
Cubic aggregate	$IP/IR = \text{Lambda}^3 \pmod p$ OK	$IQ/Iq = \text{Lambda}^3 \pmod N$ FAILS
Carry	$a3(x), b3_y(y)$ on p	MISSING $b3_x, b3_yN$ on N
Range delta	fixed p-N	only global $\text{Lambda}^*(p-N)$

3. Core constants

$\text{delta} = p - N = 432420386565659656852420866390673177326$

$\text{Lambda (p-side bridge)} = 97451685862885086182458552040892158509924235661624603229050850812487253689501$

$\text{Lambda}_N = Px3/rx3 \pmod N = 107329397079532295557318141750802481743365519021307735330654746890138631346025$

$\text{GAP} = \text{Lambda}_N - \text{Lambda} \pmod N = 9877711216647209374859589709910323233441283359683132101603896077651377656524$

$Px3/rx3 \pmod p == \text{Lambda} ? \text{ True}$
 $Px3/rx3 \pmod N == \text{Lambda} ? \text{ False}$
 $Qx3 == \text{Lambda} * qx3 \pmod N ? \text{ False}$
 $Qx3 == \text{Lambda}_N * qx3 \pmod N ? \text{ True}$

4. N-side BRIDGE block (to add)

BRIDGE_N:
 $Qx_i / qx_i = CQ1/Cq1 = \text{Lambda}_N \pmod N$
 $Qx_i = \text{Lambda}_N * qx_i \pmod N$

Where:
 $Qx = Px * \text{delta}, \quad qx = rx * \text{delta}, \quad \text{delta} = p - N$
 $\text{Lambda}_N = \text{Lambda} + \text{GAP}$
 $\text{GAP} = Px3 * rx3^{-1} - \text{Lambda} \pmod N$

NOT Lambda alone. The p constant is the shadow; Lambda_N is the live N bridge.

5. Three corner deltas (Puzzle 135 lane)

Dual bands: d in $[2^{134}, 2^{135}-1]$; mirror $[N-(2^{135}+1), N-2^{134}]$. Mirror ceiling is +2 below naive $N-(2^{135}-1)$. No delta

at $N+2^{134}$ (outside the lane; that anchor is not used).

```
delta_B = p - (N - 2^135)      = 43988563352445782980164370617657004243694
delta_C = p - (N - 2^134)      = 22210491869505721318508395742023838710510
delta_D = p - (N - (2^135+1))  = 43988563352445782980164370617657004243695
```

```
G_low      = (N - 2^134) - 2^134 = N - 2^135 =
115792089237316195423570985008687907809281421313194781059293213390251830427969
G_high     = mirror_lo - TOP      = N - 2^136 =
115792089237316195423570985008687907765725278347314657735981263638985499361601
```

Interior (if d known):

```
defect(d) = delta + d = p - (N - d)
new_N(d) = N - d
```

Relative offset (gap delta minus defect):

```
at floor d=2^134: offset = 2^134
at ceiling via G_high: offset = 2^135 + 1
delta_D - defect(TOP) = 2 (mirror +2)
```

6. Same make-up: x^3 (3 branches) vs y^2 (2 branches)

The bridge follows the same normalization pattern on x and y, but the algebraic degree fixes the branch count. x^3 gives three cube-root slots (n_1, n_2, n_3) and the A/B/C Latin square. y^2 gives only two square-root branches (y and -y). There is no third y slot analogous to n_3 .

x-side (cubic, 3 branches):

```
n^3 = N (mod p) -> n1, n2, n3
Gx_i * n_j^-1, Px_i * n_j^-1, rx_i * n_j^-1 -> Latin square A/B/C
Px3 * n1^-1 = P_B, rx3 * n1^-1 = r_B (branch-3 slot pinned)
Px3/rx3 = Lambda (one x-ratio, collapsed)
```

y-side (quadratic, 2 branches only):

```
y^2 = x^3 + 7 (mod p) -> exactly (y, -y) per x
NO Py3 from n3 normalization -> only (Py+, Py-) and (ry+, ry-)
Complexity files define Px3 and rx3 only; y is not a collapsed constant like Lambda
```

External anchor (not from complexity file): published puzzle pubkey picks one Py branch (0x667a05e9... even y). That does not create a third cube root; it only selects one member of the +/- pair.

7. Reflection pairing (y bridge rule)

y branches are coupled, not independent. The reflection bridge maps one y branch to N and pairs it with its negation:

```
p-side: y + (-y) = p
N-side: lambda_y = floor(N * y / p) mod N
        lambda_{-y} = floor(N * (-y) / p) mod N
        lambda_y + lambda_{-y} = N (or N-1 / N+1 from floor)
```

P (Px3):

```
y+      = 69440582532487379038177105219556131878098716909078780397935812212458092996863
y-      = 46351506704828816385393879789131775975171267756561783641521771795450741674800
lam_y   = 69440582532487379038177105219556131877839394154477632330194577514792613597351
lam_-y  = 46351506704828816385393879789131775974998170124597272052410585626725547896985
sum     = 115792089237316195423570985008687907852837564279074904382605163141518161494336
PASS    = True
```

r (rx3):

```
y+      = 49714739208247555872780528359092797866261457510155690641636464864972500227644
y-      = 66077350029068639550790456649595109987008527155484873397821119142936334444019
lam_y   = 49714739208247555872780528359092797866075800036592174229586386865488945900239
lam_-y  = 66077350029068639550790456649595109986761764242482730153018776276029215594097
sum     = 115792089237316195423570985008687907852837564279074904382605163141518161494336
PASS    = True
```

Reflection fixes the partner branch: choose y, then -y is determined. This is the y-side analog of x-side collapse, but with 2 branches instead of 3.

8. Lambda_x / Lambda_y and 2x2 branch table

lambda_x = Px/rx3 = Lambda (unique, from cubic lane). lambda_y = Py/ry depends on which square-root branch is taken on each side. Four pairings collapse to two distinct lambda_y values.

Naive (WRONG): lambda_x^3 + 7 = lambda_y^2 (mod p)
Correct: lambda_y^2 = (Px^3 + 7) / (rx^3 + 7) (mod p)

Pairing	lambda_y (mod p)	same parity?
Py+ x ry+	2305535029389476561013708450661632864805639220643918499	Yes
Py+ x ry-	9273673894342142981343390050207157920521359245920137904	Yes
Py- x ry+	9273673894342142981343390050207157920521359245920137904	Yes
Py- x ry-	2305535029389476561013708450661632864805639220643918499	Yes

Distinct lambda_y values across 4 pairings: 2 (not 4)
Opposite-parity pairings share one value; same-parity share the other.

Example (pubkey Py+ x ry+):
lambda_y = 92736738943421429813433900502071579205213592459201379042094542895571506924317
lambda_y == Lambda ? False
k = lambda_y/lambda_x mod p = 96896380249913414475490055076671490450009407789240445582882446462268000724666

POWER (mod p), any pairing:
lambda_y^2 == (Px^3+7)/(rx^3+7) ? True for all 4 pairings
lambda_x^3 + 7 == lambda_y^2 ? False

N-side (example pairing, mod N):
Qx3 == Lambda_N * qx3 ? True
lambda_yN^2 == (Qx^3+7)/(qx^3+7) ? False
lambda_xN^3 + 7 == lambda_yN^2 ? False

IP/IR = Lambda^3 is an x/cubic artifact only. y needs its own ratio lambda_y per branch pairing plus reflection.
Lambda^3 cube roots do not transfer to N.

9. Heaven carry (p) and N gap

p-side x carry:
a3 = (Lambda*rx3 - Px3) // p = 21882023022690643225957990962827778287572171943979549177037792017938914896823

p-side y carry (per branch pairing, example Py+ x ry+):
b3_y = (lambda_y*ry - Py) // p =
39816042891725872212869392833839431381563126813263455377457272600530356936996
Lambda*ry+ - Py+ divisible by p ? False
lambda_y*ry+ - Py+ divisible by p ? False

N-side carry (not yet in N-file):
b3_x = (Lambda_N*qx3 - Qx3) // N
b3_yN = (lambda_yN*q(ry) - q(y)) // N [per chosen y branch]

Integer lift across p closes x; y needs parallel lift per (+/-) pairing.

10. Cubic aggregate failure on N

IP/IR mod p == Lambda^3 mod p ? True
IQ/Iq mod N == Lambda^3 mod N ? False
IQ/Iq mod N == Lambda_N^3 mod N ? False

N-file lists Lambda^3*(p-N) but never closes IQ/Iq. Cubic on N needs a range-aware replacement, not a direct copy of p-side Lambda^3.

Why Lambda^3 cube roots do not transfer:
- x cubic IP/IR = Lambda^3 uses only lambda_x
- y ratio lambda_y != lambda_x; power law is (Px^3+7)/(rx^3+7), not lambda_x^3+7
- mod N, even the ratio power law fails on scaled coords until defect(d) is applied

11. Orphans in N-file (unconnected)

```
Cq = RQ*Rq^-1 mod N = 3820628127091453859030266576898546114566560342084415068589713593856641559477
equals Lambda? False
equals GAP? False
equals Lambda_N? False
```

```
p3      defect      root      branch:      C3      =      Px3/p3      mod      N      =
31364000942182736358370866274770775676862698943039335528366879013151855459557
```

These aggregates exist in N-file but are not wired to Lambda_N or to d in $[2^{134}, 2^{135})$.

12. Scalar reduction (tool note)

Separate from the bridge files but relevant to multiplier checks: correct rule is $(k \bmod N) \cdot G$. Example $k = 16 \cdot (N+1) = 16N+16$ has $k \bmod N = 16$, so $k \cdot G = 16G$. Tools that treat $k \geq N$ as infinity in widening blocks $(2N, 4N, 8N, 16N)$ are wrong; only multiples of N exactly map to O .

13. Closure sentence

```
x: 3 branches (x^3, n1/n2/n3) -> Lambda = Px/rx (mod p), Lambda_N (mod N)
y: 2 branches (y^2) -> (Py+, Py-), (ry+, ry-); lambda_y + lambda_-y = N
y power: lambda_y^2 = (Px^3+7)/(rx^3+7), NOT lambda_x^3+7
Height: defect(d) = delta + d, new_N(d) = N - d
Scalar: only (k mod N) matters; 16(N+1) -> residue 16
```

14. Priority actions

1. Add BRIDGE_N with Lambda_N (x) and reflection pair lambda_y / lambda_-y (y).
2. Document x^3 (3-way) vs y^2 (2-way) branch structure in Complexity_Simplified_N.
3. Add three corner deltas (B/C/D) and $\text{defect}(d) = \text{delta} + d$ for Puzzle 135 lane.
4. Record a_3 (x carry) and b_3_y per (+/-) pairing; derive b_3_x and b_3_yN on N.
5. Use ratio power law $(Px^3+7)/(rx^3+7)$; reject naive $\text{lambda_}x^3+7$.
6. Do not copy Lambda^3 cube-root tree to N; IQ/Iq needs height-aware replacement.
7. Regenerate anchors from Lambda_N + reflection pairing + relative offset.